

Открытое перенаправление

Описание проблемы

Веб-сайт считается уязвимым для открытой переадресации (open Redirect), если пользователь может манипулировать значением параметра (часть URL-адреса после «?») в HTTP GET-запросе позволяет перенаправить пользователя на новый сайт без проверки целевого сайта. В зависимости от архитектуры уязвимого сайта, перенаправление может произойти после определённых действий, таких как вход в систему, а иногда это может произойти мгновенно при загрузке страницы.

В Python модуль `urllib3` по умолчанию не позволяет редиректы. Но можно использовать классы `HTTPConnectionPool` или `HTTPSConnectionPool` с параметром "redirect" установленным в значение `True`. Однако для редиректов чаще всего используется класс `PoolManager`, у которого встроенная функция перенаправления запросов.

Пример небезопасного кода

```
1 import urllib3
2
3 http = urllib3.PoolManager()
4
5 # получаем URL в качестве входного параметра
6 user_input = params.Get('url')
7
8 # из-за PoolManager редирект случится автоматически
9 response = http.request('GET', user_input)
```

В данном случае в метод `request` попадают данные, полученные из GET-параметра "url". Эти данные содержат URL, на который будет осуществляться переадресация. Злоумышленник может передать в качестве параметра следующую строку:

```
http://trusted.com/example.php?url=http://evil.site.com
```

Пользователь увидит, что ссылка ведет на доверенный сайт (trusted.com) и не будет подозревать, что он будет перенаправлен на вредоносную страницу (evil.site.com). Ссылка на перенаправляемый сайт ("<http://evil.site.com>") может быть закодирована так, что пользователь подумает, что это обыкновенный параметр URL.

Последствия эксплуатации

Чаще всего уязвимость Open Redirect используется при фишинговых атаках (злоумышленник провоцирует пользователя на переход по вредоносной ссылке).

Переходя по ссылке, пользователь может даже не догадываться, куда она перенаправит его. В результате успешной атаки злоумышленник может получить приватные данные пользователя, например пароль от его личного кабинета. Так же уязвимость может привести к XSS со всеми вытекающими последствиями.

Как исправить?

Лучше всего не использовать механизмы, позволяющие автоматический редирект. Однако, если это необходимо с точки зрения бизнес-процессов, то используйте белый список. В зависимости от задачи, это может быть список разрешенных URL или разрешенных параметров URL, как в примере ниже.

Пример безопасного кода

```
1 import urllib3
2
3 http = urllib3.PoolManager()
4
5 # задаем белый список
6 whitelist = ('value1', 'value2', 'value3', 'value4')
7
8 # получаем URL в качестве входного параметр
9 input_param = params.Get('something')
10
11 # запрос выполнится только если входной параметр в белом списке
12 if input_param in whitelist:
13     url_redirect = url + input_param
14     response = http.request('GET', url_redirect)
```

Дополнительно

1. Реализовать подтверждение перехода. Необходимо предупреждать пользователя о том, что он покидает сайт, и совершать переход только после подтверждения перехода.
2. Использовать в URL вместо фактических значений параметров различные идентификаторы (числовые, guid).

Полезные ссылки

1. https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html
2. <https://cwe.mitre.org/data/definitions/601.html>